

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Приглашенный преподаватель НИУ
ВШЭ

_____ С. Б. Гурин
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Программная инженерия», старший
преподаватель департамента
программной инженерии

_____ Н. А. Павлочев
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ МЕЖДУНАРОДНЫХ ЗВОНКОВ ЧЕРЕЗ БРАУЗЕР

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.12.18-01 81 01-1-ЛУ

Исполнитель:

Студент группы БПИ243

_____ / Н. Д. Смирнов, Р. А. Николаев, Г. Д. Воронин /
«__» _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл	

УТВЕРЖДЕН

RU.17701729.12.18-01 81 01-1-ЛУ

ПЛАТФОРМА ДЛЯ МЕЖДУНАРОДНЫХ ЗВОНКОВ ЧЕРЕЗ БРАУЗЕР

Пояснительная записка

RU.17701729.12.18-01 81 01-1

Листов 33

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Данный программный документ представляет собой пояснительную записку к программному проекту «Платформа для международных звонков через браузер» (Browser International Calls Platform) и содержит следующие разделы: «Введение», «Назначение разработки», «Технические характеристики», «Ожидаемые технико-экономические показатели», приложения.

Раздел «Введение» включает в себя наименование программы на русском и английском языках, а также документ, на основании которого ведётся разработка, с указанием организации, утвердившей данный документ.

В разделе «Назначение разработки» указаны функциональное и эксплуатационное назначение создаваемого программного продукта, а также краткая характеристика области его применения.

В разделе «Технические характеристики» присутствуют следующие подразделы: постановка задачи на разработку программы, описание и обоснование выбора состава технических и программных средств, описание и обоснование архитектуры программы, описание функционирования программы, описание и обоснование выбора метода организации входных и выходных данных, описание работы с базой данных.

В разделе «Ожидаемые технико-экономические показатели» указаны ориентировочная экономическая эффективность, предполагаемая потребность и экономические преимущества разработки по сравнению с аналогами.

Настоящий документ разработан в соответствии с требованиями: 1.ГОСТ 19.101-77 [1]: Виды программ и программных документов. 2.ГОСТ 19.102-77 [2]: Стадии разработки. 3.ГОСТ 19.103-77 [3]: Обозначения программ и программных документов. 4.ГОСТ 19.104-78 [4]: Основные надписи. 5.ГОСТ 19.105-78 [5]: Общие требования к программным документам. 6.ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом. 7.ГОСТ 19.404-79 [10]: Пояснительная записка. Требования к содержанию и оформлению.

Изменения к данной Пояснительной записке оформляются согласно ГОСТ 19.603-78 [12], ГОСТ 19.604-78 [13].

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Документ(ы), на основании которого(ых) ведется разработка	4
1.3. Способ применения моделей	4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение	5
2.3. Краткая характеристика области применения программы	5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	7
3.1. Постановка задачи на разработку программы	7
3.2. Описание и обоснование выбора состава технических и программных средств	8
3.2.1. Состав технических и программных средств	8
3.2.2. Обоснование выбора технических и программных средств	9
3.3. Описание и обоснование архитектуры программы	11
3.4. Описание функционирования программы	13
3.5. Описание и обоснование выбора метода организации входных и выходных данных	21
3.6. Описание работы с базой данных	23
3.7. Способ применения моделей	24
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	26
4.1. Ориентировочная экономическая эффективность	26
4.2. Предполагаемая потребность	26
4.3. Экономические преимущества разработки по сравнению с аналогами	26
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	28
ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ	30
ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ	31

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы на русском языке — «Платформа для международных звонков через браузер».

Наименование программы на английском языке — «Browser International Calls Platform».

1.2. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

1.3. Способ применения моделей

Настоящий документ содержит раздел «Способ применения моделей», в котором описывается использование генеративных моделей искусственного интеллекта (Claude Codex и Cline с DeepSeek provider API) при разработке клиентской части программы.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Программа предназначена для обеспечения возможности совершения международных телефонных звонков через веб-браузер без необходимости установки дополнительного программного обеспечения. Основные функции:

- Регистрация и аутентификация пользователей по email и паролю с выдачей JWT-токена;
- Выбор страны из предопределенного списка и набор телефонного номера через веб-интерфейс;
- Совершение звонков с использованием технологии WebRTC через VoIP-сервис (Twilio);
- Отображение статуса звонка в реальном времени (соединение / разговор / завершен);
- Просмотр истории совершенных звонков с информацией о дате, времени, номере, длительности и статусе;
- Автоматическая проверка доступа к микрофону и динамикам устройства при входе на экран звонка;
- Отображение ошибок соединения в понятном для пользователя виде;
- Локализация интерфейса (русский и английский языки) с возможностью переключения;
- Адаптивный интерфейс, корректно отображающийся на десктопных и мобильных браузерах.

2.2. Эксплуатационное назначение

Основной аудиторией платформы являются:

- Частные лица, нуждающиеся в быстрой связи с родственниками и друзьями за рубежом без установки дополнительного программного обеспечения;
- Малый и средний бизнес для осуществления международных коммуникаций;
- Пользователи с базовыми навыками работы с браузером, которым важно, чтобы сервис работал «из коробки».

2.3. Краткая характеристика области применения программы

«Платформа для международных звонков через браузер» — это клиент-серверное веб-приложение, предназначенное для организации двусторонних голосовых звонков между пользователями через веб-браузер и телефонные сети общего пользования. Приложение обеспечивает:

- Организацию голосовых звонков через веб-интерфейс без установки дополнительного ПО;
- Интеграцию с телефонными сетями для совершения международных звонков через облачный VoIP-сервис Twilio;
- Поддержку двусторонней голосовой связи между браузером и телефонным абонентом по

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

протоколу WebRTC;

- Управление звонками и просмотр истории через единый веб-интерфейс.

Программа может использоваться на любых устройствах с современным веб-браузером (Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge актуальных версий) и доступом в интернет: персональных компьютерах (Windows, macOS, Linux), смартфонах и планшетах (iOS, Android). Область применения включает телекоммуникации, интернет-телефонию, бизнес-коммуникации, удаленную работу и образовательные проекты.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Разрабатываемое приложение должно быть реализовано как клиент-серверное веб-приложение. Клиентская часть реализуется на языке TypeScript с использованием библиотеки React и собирается инструментом Vite. Серверная часть реализуется на языке Go. В рамках реализации необходимо включить следующие модули:

1. Управление пользователями

1.1. Регистрация пользователя по email и паролю с валидацией формата email по стандарту RFC 5322.

1.2. Аутентификация пользователя с выдачей JWT-токена, подписанного алгоритмом HS256.

1.3. Выход из системы с инвалидацией сессии на клиентской стороне.

2. Совершение звонков

2.1. Выбор страны из предопределенного списка с отображением флага, названия и международного телефонного кода.

2.2. Ввод номера телефона с клиентской валидацией (минимум 5 цифр) и автоустановкой префикса-кода страны.

2.3. Инициация звонка через REST API с последующей установкой WebRTC-соединения с использованием Twilio Voice SDK.

2.4. Завершение звонка с автоматическим расчетом и сохранением длительности в секундах.

2.5. Отображение статуса звонка в реальном времени: «соединение» (initiated), «соединение...» (connecting), «разговор» (active), «завершен» (completed).

3. История звонков

3.1. Просмотр списка последних звонков текущего пользователя с информацией: номер телефона, дата и время начала, длительность, статус.

3.2. Автоматическая фильтрация истории по идентификатору аутентифицированного пользователя.

4. Обработка устройств

4.1. Автоматическая проверка доступа к микрофону и аудиовыходу при загрузке экрана звонка через WebRTC API (getUserMedia).

4.2. Отображение предупреждений при отсутствии доступа к устройствам с инструкцией по настройке браузера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. Интерфейс

5.1. Адаптивный веб-дизайн, обеспечивающий корректное отображение на десктопных и мобильных браузерах.

5.2. Локализация интерфейса на русский и английский языки с сохранением выбранной локали между сессиями.

5.3. Защищенные маршруты: доступ к экранам звонка и истории только после успешной аутентификации, с автоматическим перенаправлением неавторизованных пользователей на лендинг.

6. Безопасность

6.1. Передача данных между клиентом и сервером по протоколу HTTPS.

6.2. Хеширование паролей алгоритмом bcrypt с автоматической генерацией соли.

6.3. Stateless-аутентификация через JWT-токены, передаваемые в HTTP-заголовке Authorization: Bearer.

6.4. Валидация входных данных как на клиентской, так и на серверной стороне.

7. Инфраструктура

7.1. Контейнеризация всех компонентов (база данных, бэкенд, фронтенд) через Docker Compose с единой командой запуска.

7.2. Использование СУБД PostgreSQL 16 для хранения пользователей и истории звонков.

7.3. Автоматическое применение миграций базы данных при старте приложения.

3.2. Описание и обоснование выбора состава технических и программных средств

3.2.1. Состав технических и программных средств

Для работы приложения требуется следующее техническое и программное обеспечение:

Серверная часть:

- Сервер или виртуальная машина с установленным Docker и Docker Compose;
- Операционная система: Linux (рекомендуется) или Windows с поддержкой Docker;
- Минимальные аппаратные требования: 2 ядра CPU, 2 ГБ ОЗУ, 10 ГБ дискового пространства.

Клиентская часть:

- Любое устройство с современным веб-браузером (Google Chrome 90+, Mozilla Firefox 88+, Apple Safari 14+, Microsoft Edge 90+);
- Наличие микрофона и динамиков (или гарнитуры) для совершения звонков;
- Доступ в интернет со скоростью не менее 1 Мбит/с для обеспечения качества голосовой связи;
- Операционные системы клиента: Windows 10+, macOS 11+, Linux, iOS 14+, Android 10+.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Состав программных средств разработки и эксплуатации:

Клиентская часть (фронтенд):

- Язык программирования: TypeScript 5.9;
- Библиотека пользовательского интерфейса: React 19.2;
- Инструмент сборки: Vite 7.2;
- Стилизация: CSS Modules;
- WebRTC API браузера (нативный);
- Twilio Voice SDK 2.16 для интеграции с VoIP-сервисом;
- React Router DOM 7.13 для маршрутизации;
- Обратный прокси-сервер: Nginx (для раздачи статических файлов и проксирования API-запросов).

Серверная часть (бэкенд):

- Язык программирования: Go 1.23;
- HTTP-фреймворк: Gin v1.9;
- ORM-библиотека: GORM v1.25 с драйвером PostgreSQL;
- Аутентификация: библиотека golang-jwt/jwt/v5 для работы с JWT-токенами;
- Хеширование паролей: golang.org/x/crypto (bcrypt);
- VoIP-интеграция: Twilio Go SDK (twilio-go v1.20).

База данных:

- СУБД: PostgreSQL 16 (образ postgres:16-alpine в Docker);
- Драйвер для Go: GORM PostgreSQL driver v1.5.

Инфраструктура:

- Контейнеризация: Docker Engine + Docker Compose;
- Обратный прокси: Nginx (встроен в контейнер фронтенда).

3.2.2. Обоснование выбора технических и программных средств

TypeScript выбран для клиентской части, так как является строго типизированным надмножеством JavaScript. Статическая типизация позволяет обнаруживать ошибки на этапе компиляции, улучшает автодополнение в IDE и повышает общую надежность кодовой базы, что особенно важно при работе с асинхронными операциями WebRTC.

React 19 выбран в качестве библиотеки для построения пользовательского интерфейса благодаря компонентному подходу, высокой производительности за счет Virtual DOM и обширной

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

экосистеме. Функциональные компоненты с хуками позволяют создавать переиспользуемую и легко тестируемую логику.

Vite используется как современный инструмент сборки, обеспечивающий практически мгновенный запуск dev-сервера (благодаря нативным ES-модулям) и быструю горячую замену модулей (HMR), что значительно ускоряет процесс разработки.

Go (Golang) 1.23 выбран для серверной части благодаря следующим преимуществам: высокая производительность за счет компиляции в машинный код, статическая типизация, встроенная поддержка конкурентности через горутин (goroutines), удобное развертывание в виде единого бинарного файла. Эти качества делают Go оптимальным выбором для высоконагруженных API-сервисов, обрабатывающих множество одновременных соединений.

Gin v1.9 — легковесный и высокопроизводительный HTTP-фреймворк для Go, обеспечивающий быструю маршрутизацию, middleware-архитектуру и удобную работу с JSON. Gin входит в число самых быстрых Go-фреймворков по результатам бенчмарков благодаря использованию httprouter.

GORM v1.25 обеспечивает удобную работу с базой данных через ORM-подход: автоматические миграции схемы, построение запросов на основе Go-структур, защиту от SQL-инъекций через параметризованные запросы. Выбор GORM обоснован его зрелостью и широкой распространенностью в Go-сообществе.

PostgreSQL 16 выбрана как надежная реляционная СУБД с открытым исходным кодом. Ключевые преимущества: полная поддержка ACID-транзакций, эффективная работа с JSON, расширяемость, активное сообщество. Для контейнеризованной разработки используется облегченный образ postgres:16-alpine.

JWT (JSON Web Token) обеспечивает stateless-аутентификацию, удобную для распределенных систем и API-сервисов. Отсутствие необходимости хранить сессии на сервере снижает нагрузку и упрощает горизонтальное масштабирование. Алгоритм HS256 выбран как стандартный алгоритм на основе HMAC-SHA256.

Bcrypt является стандартом де-факто для защищенного хеширования паролей. Алгоритм автоматически генерирует случайную соль и имеет настраиваемую вычислительную сложность (cost factor), что делает его устойчивым к атакам перебором и радужными таблицами.

Twilio предоставляет готовый облачный API для интеграции с телефонными сетями общего пользования (PSTN). Twilio Voice SDK поддерживает WebRTC, что позволяет устанавливать прямые

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

медиа-соединения между браузером и телефонным абонентом. Использование готового VoIP-сервиса устраняет необходимость в разработке и поддержке собственной сложной инфраструктуры телефонии.

Docker Compose обеспечивает простое развертывание всего стека (база данных, бэкенд, фронтенд) одной командой `docker-compose up -d`. Контейнеризация изолирует окружение, устраняет проблемы несовместимости версий и упрощает развертывание на различных средах.

Nginx используется как обратный прокси-сервер внутри контейнера фронтенда для объединения клиентской и серверной частей на едином порту (1573), а также для эффективной раздачи статических файлов с использованием асинхронной событийно-ориентированной архитектуры.

3.3. Описание и обоснование архитектуры программы

Приложение построено по архитектурному паттерну «клиент-сервер». Клиентская часть реализована на React с компонентным подходом, серверная — на языке Go с использованием Clean Architecture (Чистой архитектуры), предложенной Робертом Мартином.

Слои серверной архитектуры (Clean Architecture):

1. Domain Layer (internal/domain) — доменные модели и интерфейсы:

Содержит определения сущностей User (пользователь: ID, Email, PasswordHash, CreatedAt) и Call (звонок: ID, UserID, PhoneNumber, StartTime, Duration, Status, SessionID, SDPOffer, SDPAnswer). Включает интерфейсы репозитория (UserRepository, CallRepository) и VoIP-сервиса (VoIPService). Данный слой не зависит от других слоев и внешних библиотек.

2. Use Cases Layer (internal/use_cases) — бизнес-логика приложения:

Содержит реализации сценариев использования: auth/ (RegisterUseCase, LoginUseCase, LogoutUseCase) — регистрация, аутентификация и выход; calls/ (StartCallUseCase, EndCallUseCase, InitiateCallUseCase, TerminateCallUseCase) — инициация и завершение звонков; history/ (ListHistoryUseCase) — получение истории звонков. Каждый use case принимает входную структуру (Input) и возвращает выходную (Output).

3. Infrastructure Layer (internal/infrastructure) — реализация интерфейсов:

- postgres/ — реализация UserRepository и CallRepository через GORM с пулом соединений (максимум 25 открытых, 5 простаивающих);
- jwt/ — сервис генерации и валидации JWT-токенов с использованием HMAC-SHA256;
- voip/ — клиент Twilio для инициации и завершения звонков, генератор Voice-токенов для клиентского SDK.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. Transport Layer (internal/transport/http) — HTTP API:

Реализует REST API на фреймворке Gin. Включает роутер с эндпоинтами (/auth/register, /auth/login, /auth/logout, /calls/initiate, /calls/terminate, /calls/history, /calls/start, /calls/end, /system/health), обработчики HTTP-запросов (AuthHandler, CallsHandler, WebRTCHandler, VoiceHandler, HistoryHandler), а также middleware для JWT-аутентификации, CORS и обработки паник.

5. Application Layer (internal/app) — композиция и инициализация:

Осуществляет связывание всех компонентов в порядке: подключение к БД → автоматические миграции → создание репозиториев → инициализация JWT-сервиса → инициализация VoIP-клиента → создание use cases → создание обработчиков → настройка роутера.

6. Configuration Layer (internal/config) — загрузка конфигурации из переменных окружения с поддержкой .env-файлов через библиотеку godotenv.

Клиентская архитектура (React):

Фронтенд построен на функциональных компонентах React с использованием хуков:

- Страницы (pages/): Landing (приветственная страница для неавторизованных пользователей), Login (форма входа), Register (форма регистрации), Call (экран звонка с выбором страны, вводом номера и элементами управления), History (экран истории звонков);

- Компоненты (components/): Layout (общий макет с верхней навигационной панелью), CountrySelect (выпадающий список стран с флагами и кодами), ProtectedRoute (защищенный маршрут, перенаправляющий неавторизованных пользователей);

- Хуки (hooks/): useDevices — автоматическая проверка доступности микрофона и динамиков через navigator.mediaDevices.getUserMedia; useWebRTC — управление WebRTC-соединением через Twilio Voice SDK;

- Контекст (contexts/): AuthContext — управление состоянием аутентификации, хранение и обновление JWT-токена в localStorage;

- Интернационализация (i18n/): LocaleContext — контекст для переключения и хранения текущей локали; translations — словари строк на русском и английском языках;

- API-клиент (api/client.ts) — централизованный модуль для HTTP-запросов к бэкенду с автоматическим добавлением JWT-токена в заголовок Authorization.

Взаимодействие между клиентом и сервером осуществляется по протоколу HTTP/HTTPS через REST API. Клиент отправляет запросы к API бэкенда в формате JSON. Аутентификация выполняется через JWT-токены, передаваемые в HTTP-заголовке Authorization: Bearer, что соответствует схеме

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

bearerAuth спецификации OpenAPI. Для VoIP-звонков используется WebRTC-соединение через Twilio Voice SDK: бэкенд генерирует Voice-токен и параметры сессии, фронтенд устанавливает прямое медиа-соединение с Twilio.

Преимущества выбранной архитектуры:

1. Разделение ответственности (Separation of Concerns): Clean Architecture на бэкенде четко разделяет бизнес-логику, инфраструктуру и транспортный слой, следуя принципу инверсии зависимостей.
2. Тестируемость: каждый слой зависит только от абстракций (интерфейсов), что позволяет легко подменять реализации на mock-объекты при модульном тестировании.
3. Масштабируемость: компонентный подход на фронтенде и многослойная архитектура на бэкенде позволяют добавлять новую функциональность с минимальными изменениями существующего кода.
4. Типобезопасность: использование TypeScript на клиенте и статическая типизация Go на сервере значительно снижают количество ошибок, обнаруживаемых на этапе выполнения.
5. Простота развертывания: единый docker-compose.yml объединяет все компоненты системы в изолированный стек, запускаемый одной командой.

3.4. Описание функционирования программы

При запуске приложения неавторизованный пользователь попадает на лендинг (Landing) — приветственную страницу с кратким описанием возможностей платформы и кнопками «Войти» и «Регистрация».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

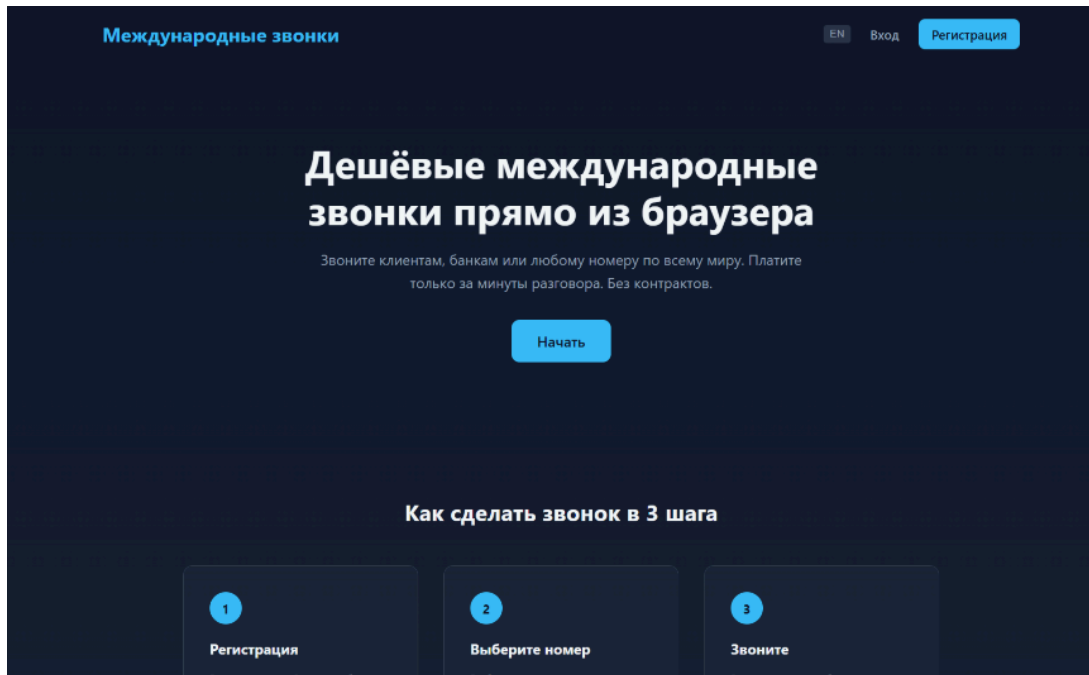
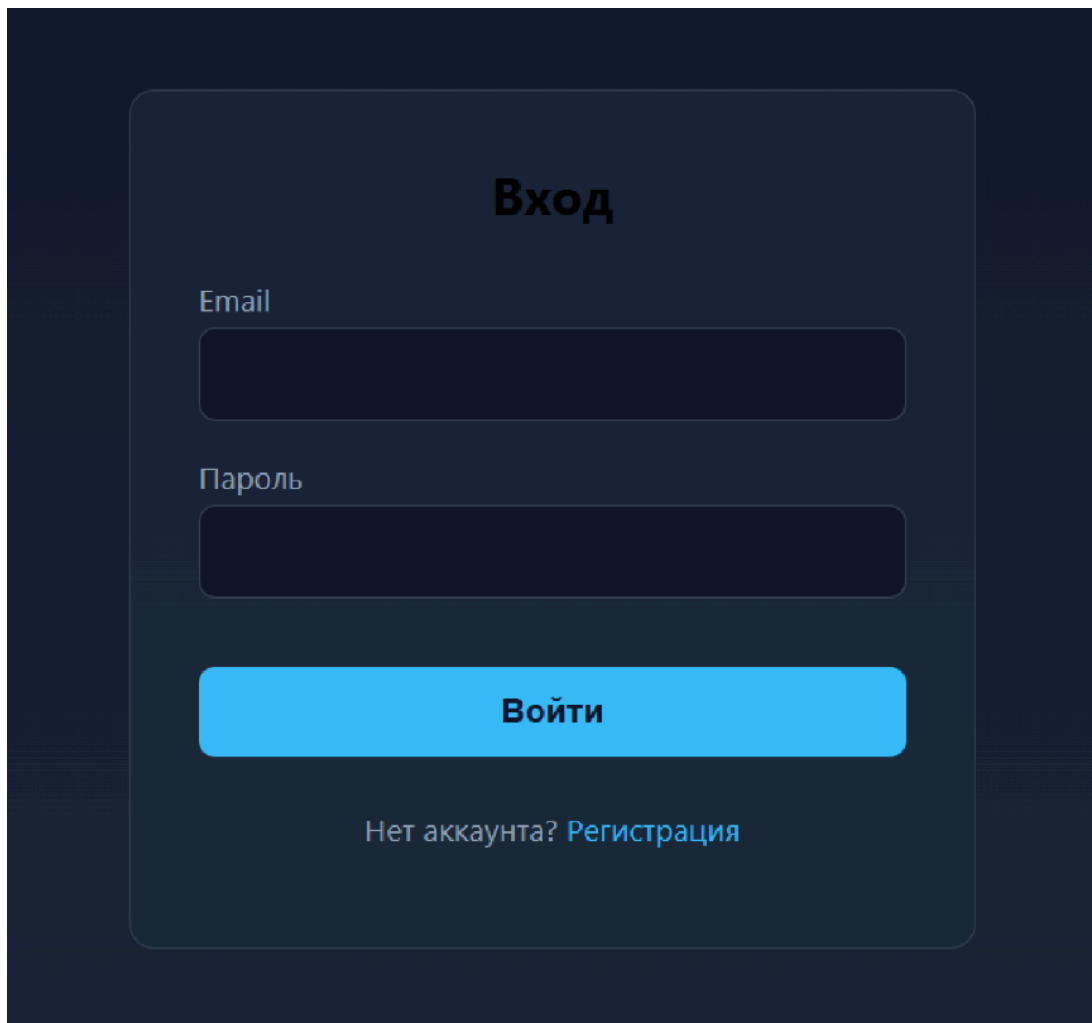


Рис. 1. Лендинг — приветственная страница

Экран входа в систему (Login): Форма входа содержит два поля: email и пароль. После ввода данных и нажатия кнопки входа клиент отправляет POST-запрос на /auth/login с JSON-телом, содержащим email и password. Сервер проверяет учетные данные: ищет пользователя по email в базе данных, сравнивает хеш пароля через bcrypt. При успешной проверке сервер генерирует JWT-токен (алгоритм HS256) и возвращает его в ответе. Токен сохраняется в localStorage браузера и используется для всех последующих защищенных запросов. При неверных данных сервер возвращает HTTP 401, и пользователю отображается сообщение об ошибке.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

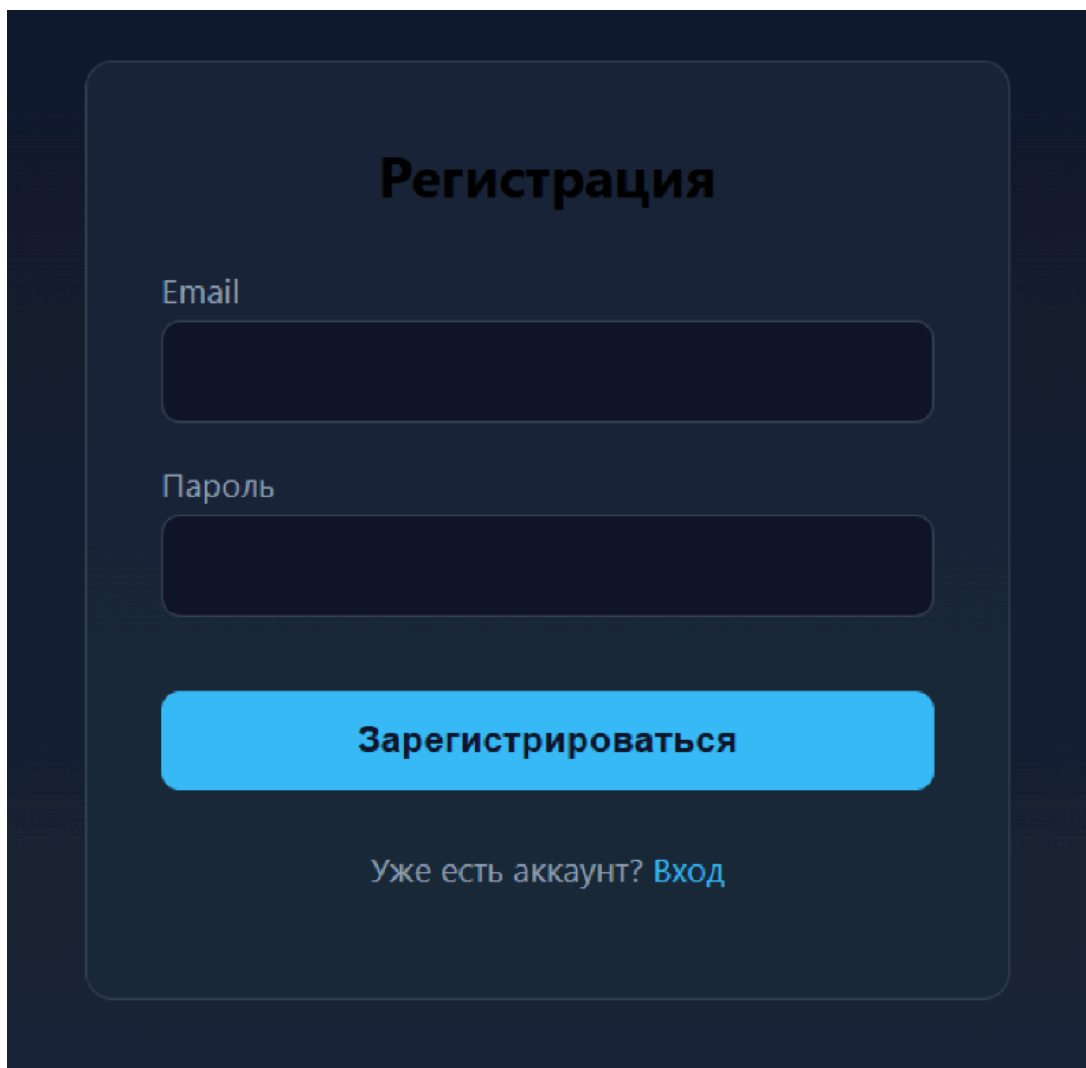


The image shows a login interface with a dark blue background. A central light blue rounded rectangle contains the title "Вход" (Login) in bold black text. Below it are two input fields: "Email" and "Пароль" (Password). The "Пароль" field has a small eye icon to toggle visibility. Below the fields is a prominent blue button with the text "Войти" (Login). At the bottom, there is a link that reads "Нет аккаунта? Регистрация" (No account? Registration).

Рис. 2. Экран входа в систему

Экран регистрации (Register): Аналогично экрану входа, форма содержит поля email и пароль. При отправке POST-запроса на /auth/register сервер проверяет уникальность email и, если пользователь не существует, создает новую запись с хешем пароля (bcrypt). После успешной регистрации (HTTP 201) пользователь перенаправляется на экран входа. При попытке регистрации с уже существующим email сервер возвращает HTTP 409.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата



Регистрация

Email

Пароль

Зарегистрироваться

Уже есть аккаунт? [Вход](#)

Рис. 3. Экран регистрации

Экран звонка (Call): При загрузке экрана звонка автоматически выполняется проверка доступа к микрофону через хук useDevices. Если доступ отсутствует, отображается предупреждение с предложением проверить настройки браузера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

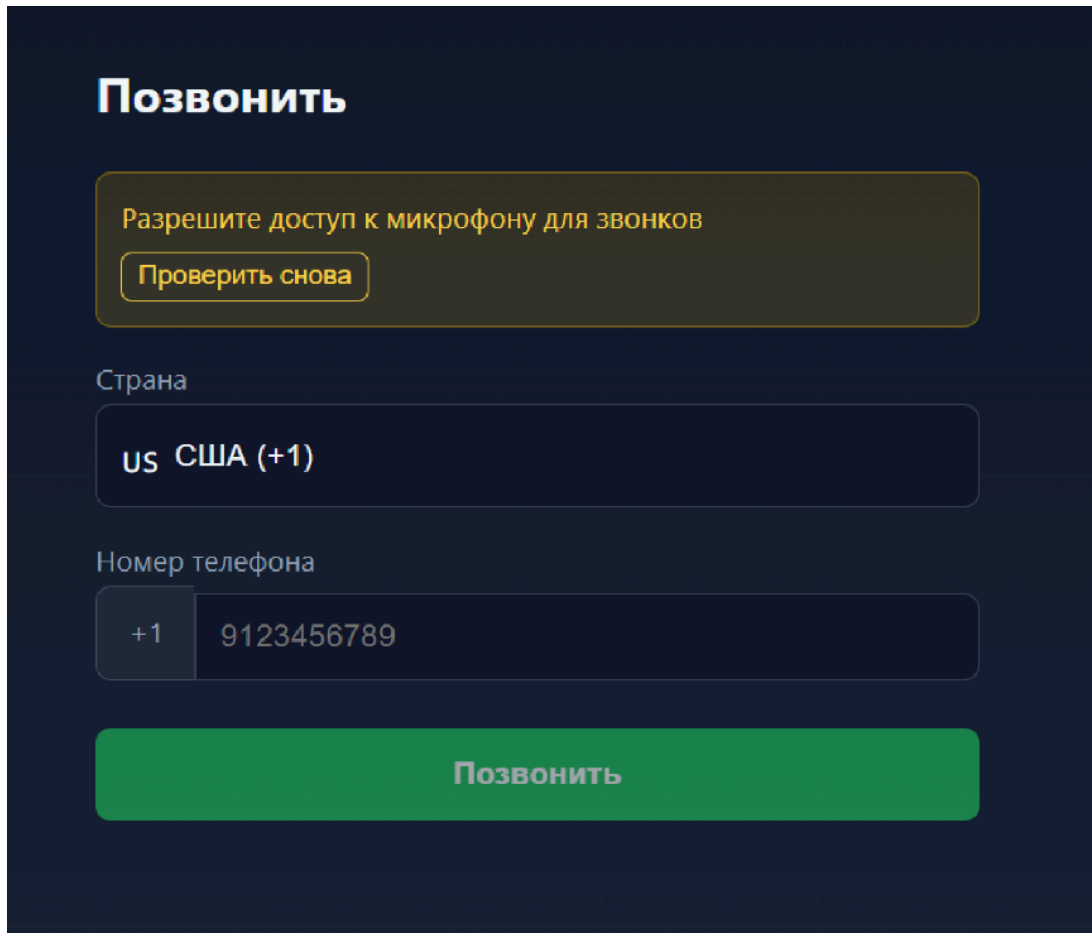


Рис. 4. Экран с предупреждением об отсутствии доступа к микрофону

В верхней части экрана расположен выпадающий список CountrySelect для выбора страны с отображением флага, названия страны и международного кода набора. Данные о странах хранятся локально в файле data/countries.ts.

Ниже расположено поле ввода номера телефона с автоматически подставленным префиксом-кодом выбранной страны. Валидация номера на клиенте требует минимум 5 цифр (исключая код страны). Кнопка совершения звонка активна только при валидном номере и доступном микрофоне.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

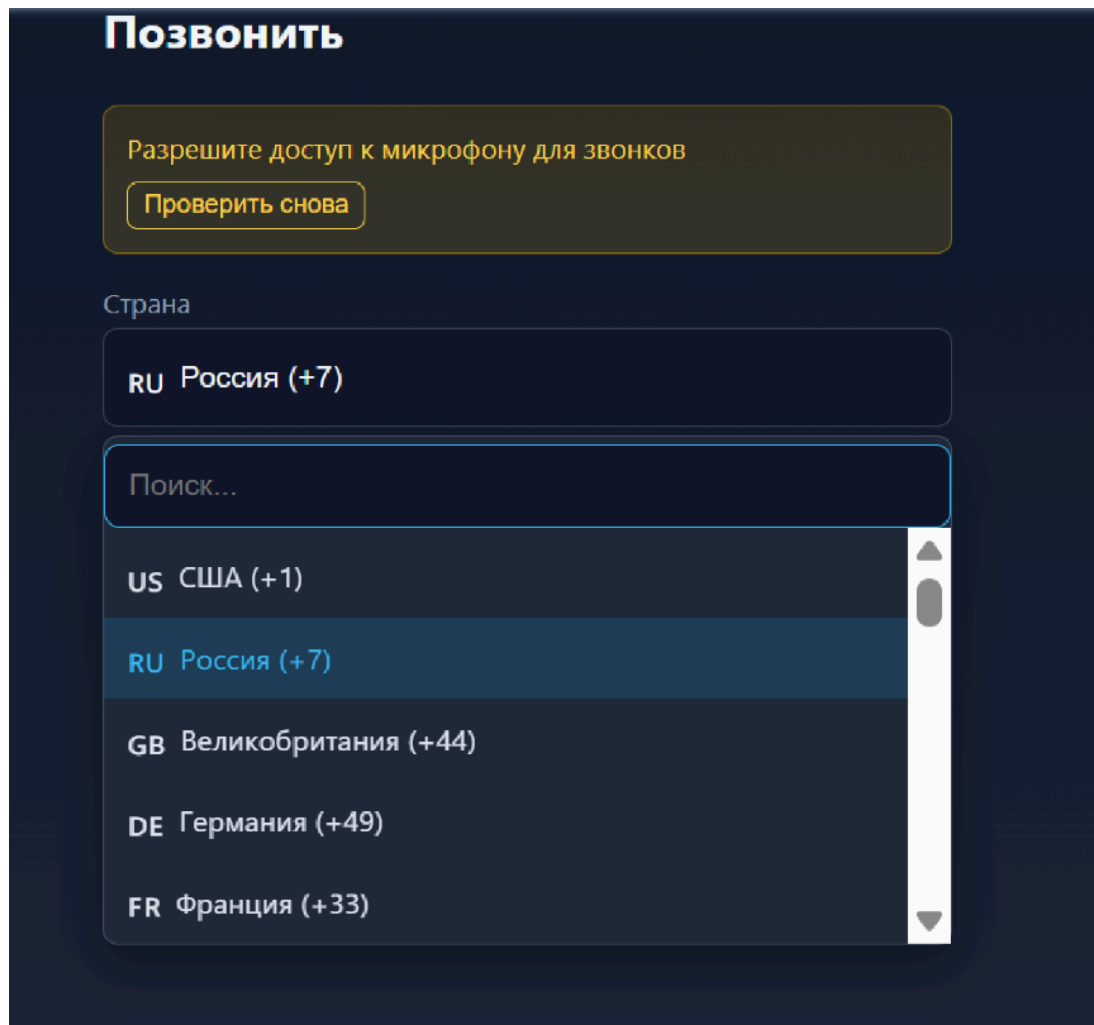


Рис. 5. Экран звонка — интерфейс выбора страны и набора номера

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

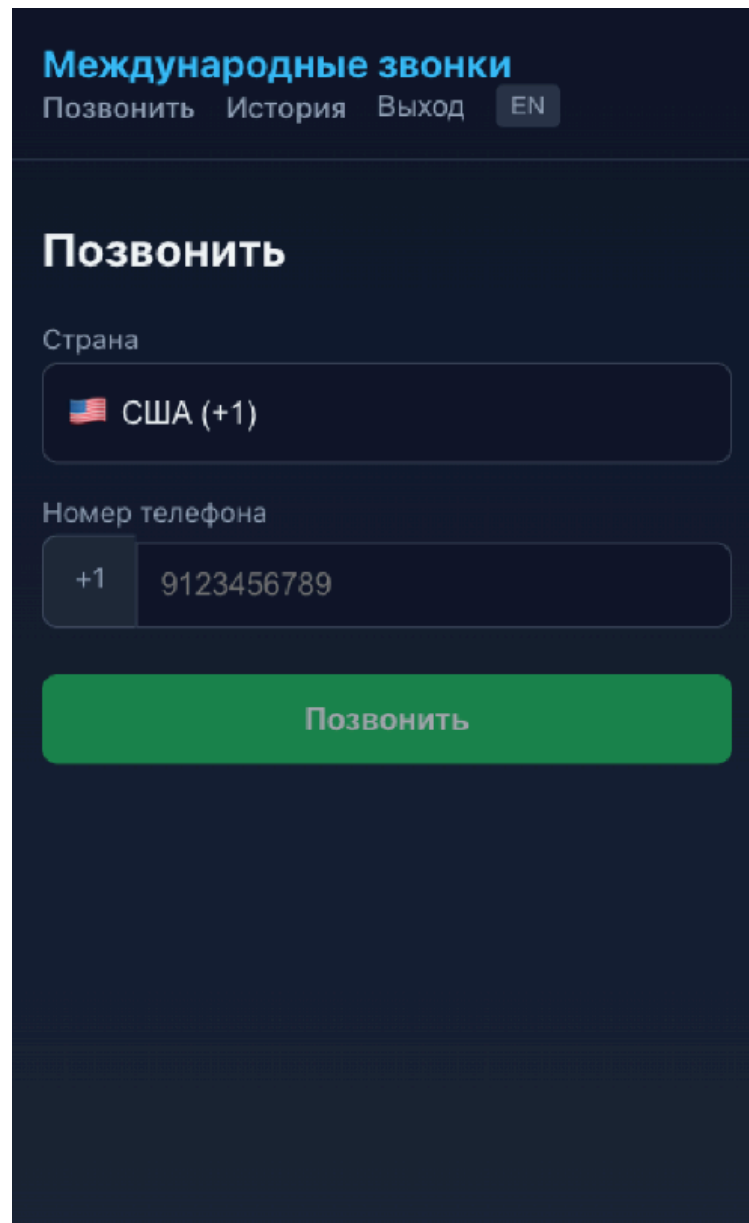


Рис. 6. Адаптивный дизайн — мобильная версия интерфейса

При нажатии кнопки звонка клиент отправляет POST-запрос на `/calls/initiate` с JSON-телом, содержащим `phone_number` в международном формате. Бэкенд создает запись звонка в БД со статусом «initiated», инициирует VoIP-сессию через Twilio API и возвращает параметры WebRTC-соединения: `call_id`, `session_id`, `sdp_offer`, статус. Фронтенд, используя Twilio Voice SDK и хук `useWebRTC`, устанавливает прямое медиа-соединение.

Аудиопоток с микрофона захватывается через WebRTC API браузера (метод `getUserMedia`). Входящий аудиопоток от собеседника воспроизводится через скрытый HTML-элемент `audio`. Управление аудиоустройствами осуществляется через WebRTC `MediaStream` API.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Во время звонка кнопка меняется на «Завершить звонок» (красная), индикатор статуса отображает текущее состояние: пульсирующая точка с текстом «Соединение...» для статусов *initiated/connecting*, зеленая точка с текстом «Разговор» для статуса *active*, серая точка с текстом «Завершен» для статуса *completed*.

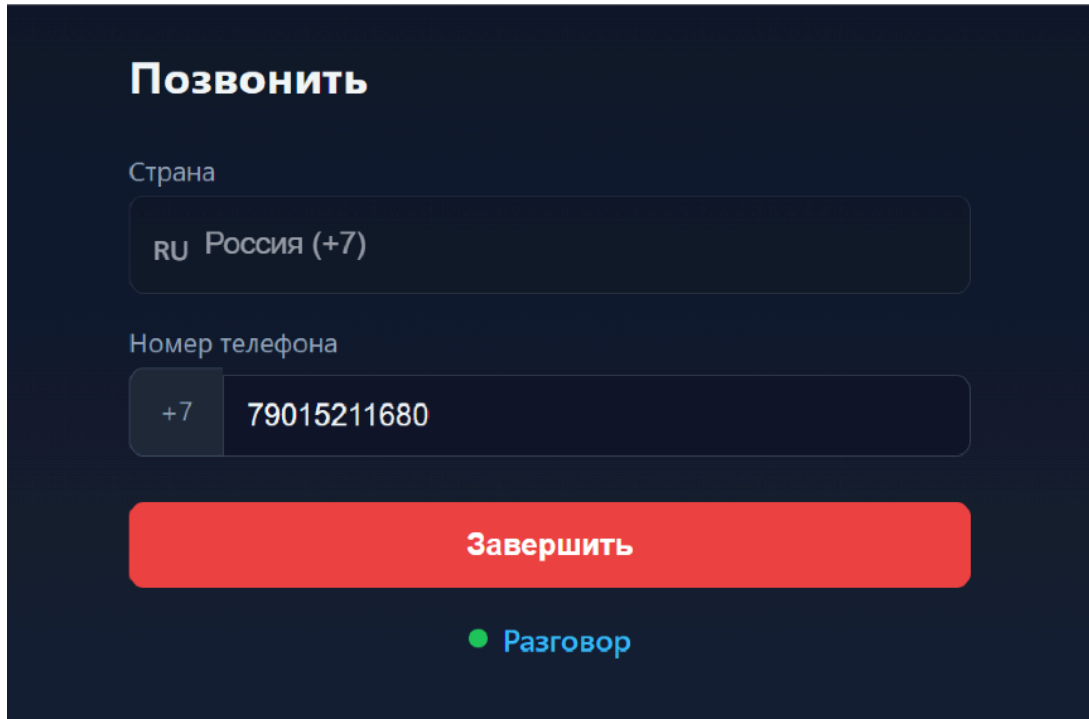


Рис. 7. Экран звонка во время активного разговора

Для завершения звонка клиент отправляет POST-запрос на `/calls/terminate` с `call_id`. Сервер завершает VoIP-сессию через Twilio API, рассчитывает длительность звонка (разница между текущим временем и временем начала) и обновляет запись в БД со статусом «completed».

Экран истории звонков (History): При переходе на экран клиент отправляет GET-запрос на `/calls/history` с JWT-токеном. Сервер извлекает `user_id` из токена и возвращает список звонков данного пользователя, отсортированный по времени начала (от новых к старым). Каждый элемент списка отображает: номер телефона, дату и время начала звонка, длительность (в секундах), статус (*completed* — завершен, *failed* — неудача, *canceled* — отменен).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

История			
Дата	Номер телефона	Длительность	Статус
11.05.2026, 21:18	+779015211680	1:04	Завершён
11.05.2026, 21:14	+779015211680	0:05	Завершён

Рис. 8. Экран истории звонков

Навигация: Верхняя навигационная панель (компонент Layout) отображается после аутентификации и содержит: название приложения, кнопки-ссылки «Звонок» и «История», кнопку «Выйти» и переключатель языка (RU/EN). Компонент ProtectedRoute защищает маршруты /call и /history, перенаправляя неавторизованных пользователей на лендинг.

Локализация: Переключение языка реализовано через LocaleContext. Все надписи интерфейса, сообщения об ошибках, статусы звонков и подписи хранятся в словарях translations.ts для русского и английского языков. Выбранная локаль сохраняется в localStorage.

Обработка ошибок: Все ошибки (сетевые, серверные, ошибки валидации) отображаются пользователю в виде текстовых сообщений в интерфейсе. Сервер возвращает ошибки в унифицированном формате JSON: {«error»: «тип_ошибки», «message»: «человекочитаемое_описание»}. HTTP-статус-коды соответствуют типу ошибки: 400 — некорректные данные, 401 — неавторизован, 403 — недостаточно прав, 404 — не найдено, 409 — конфликт, 503 — сервис недоступен.

3.5. Описание и обоснование выбора метода организации входных и выходных данных

Входные данные:

1. Email и пароль — текстовые поля в формах регистрации и аутентификации. Валидация на клиенте: email должен соответствовать формату RFC 5322, пароль должен быть непустым. Данные передаются на сервер в формате JSON: {«email»: «user@example.com», «password»: «secret»}. На сервере пароль хешируется алгоритмом bcrypt перед сохранением в БД.

2. Код страны и номер телефона — выбор страны из predetermined списка (CountrySelect) и ввод номера. Данные передаются на сервер в формате JSON: {«phone_number»: «+4915123456789»} — номер в международном формате E.164. Валидация на клиенте: минимум 5 цифр после кода страны.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. JWT-токен — передается в HTTP-заголовке Authorization: Bearer для всех защищенных энд-поинтов (/calls/initiate, /calls/terminate, /calls/history, /auth/logout). Извлекается серверным middleware и используется для идентификации пользователя.

4. Аудиопоток с микрофона — захватывается через WebRTC MediaStream API браузера (метод navigator.mediaDevices.getUserMedia с ограничением audio: true). Аудиоданные передаются напрямую на сервер Twilio через установленное WebRTC peer-соединение, минуя бэкэнд приложения.

Выходные данные:

1. Веб-страницы — пользовательский интерфейс с формами, списками, индикаторами статуса. Реализован с адаптивным дизайном через CSS Modules.

2. Статус звонка — визуальный индикатор (цветная точка с CSS-анимацией) и текстовая строка состояния, обновляемые в реальном времени.

3. История звонков — массив JSON-объектов, отображаемый в виде списка. Каждый объект содержит поля: callId (идентификатор), phoneNumber (номер телефона), startedAt (время начала в ISO 8601), durationSeconds (длительность в секундах), status (completed/failed/canceled).

4. Сообщения об ошибках — унифицированный JSON-формат с полями error (машинный код ошибки) и message (человекочитаемое описание на выбранном языке). HTTP-статус-коды соответствуют семантике ошибки.

5. Аудиопоток на динамики — принимается через WebRTC peer-соединение от серверов Twilio и воспроизводится через HTML5 Audio API (элемент audio с атрибутом autoplay).

Обоснование выбора:

JSON выбран как формат обмена данными благодаря простоте парсинга, читаемости для разработчика и нативной поддержке во всех современных языках программирования и браузерах. JSON является стандартным форматом для REST API и непосредственно поддерживается фреймворком Gin.

WebRTC обеспечивает прямую peer-to-peer передачу аудиоданных между браузером и VoIP-сервером Twilio с минимальной задержкой, что критически важно для качества голосовой связи. WebRTC является открытым стандартом, поддерживаемым всеми современными браузерами без дополнительных плагинов.

JWT обеспечивает stateless-аутентификацию: серверу не требуется хранить сессии, вся необходимая информация содержится в самом токене. Это снижает нагрузку на сервер, упрощает горизонтальное масштабирование и соответствует современной практике построения REST API.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

HTML5 Audio API используется для воспроизведения удаленного аудиопотока, так как является стандартным, поддерживаемым всеми браузерами способом работы со звуком и не требует дополнительных библиотек.

Адаптивные веб-формы с клиентской валидацией снижают количество некорректных запросов к серверу, улучшают пользовательский опыт за счет мгновенной обратной связи и соответствуют современным стандартам веб-разработки.

3.6. Описание работы с базой данных

База данных реализована на СУБД PostgreSQL 16 и содержит две основные таблицы, создаваемые через автоматические миграции при старте приложения.

Таблица users (пользователи):

Хранит учетные записи зарегистрированных пользователей. Структура:

- id — UUID, первичный ключ, генерируется автоматически функцией gen_random_uuid();
- email — VARCHAR(255), уникальный, не может быть NULL. Используется для аутентификации;
- password_hash — VARCHAR(255), не может быть NULL. Хранит результат хеширования пароля алгоритмом bcrypt;
- created_at — TIMESTAMP WITH TIME ZONE, автоматически устанавливается в CURRENT_TIMESTAMP при создании записи.

Индексы:

- idx_users_email на users(email) — ускоряет поиск пользователя по email при аутентификации и проверке уникальности.

Таблица calls (звонки):

Хранит информацию о совершенных звонках. Структура:

- id — UUID, первичный ключ, генерируется автоматически;
- user_id — UUID, внешний ключ к users(id) с каскадным удалением (ON DELETE CASCADE). Не может быть NULL;
- phone_number — VARCHAR(50), номер телефона в международном формате. Не может быть NULL;
- start_time — TIMESTAMP WITH TIME ZONE, время начала звонка. Автоматически устанавливается в CURRENT_TIMESTAMP. Не может быть NULL;
- duration — INTEGER, длительность звонка в секундах. По умолчанию 0;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- status — VARCHAR(20), статус звонка. Допустимые значения: initiated, connecting, active, completed, failed, canceled. По умолчанию «initiated». Не может быть NULL;
- created_at — TIMESTAMP WITH TIME ZONE, автоматически устанавливается при создании записи;
- session_id — VARCHAR(255), идентификатор VoIP-сессии в Twilio. Добавлен миграцией 003;
- sdp_offer — TEXT, SDP offer для установки WebRTC-соединения. Добавлен миграцией 003;
- sdp_answer — TEXT, SDP answer от Twilio. Добавлен миграцией 003.

Индексы:

- idx_calls_user_id на calls(user_id) — ускоряет фильтрацию звонков по пользователю;
- idx_calls_start_time на calls(start_time) — ускоряет сортировку звонков по времени;
- idx_calls_user_start на calls(user_id, start_time DESC) — составной индекс для эффективного получения истории звонков с сортировкой от новых к старым;
- idx_calls_session_id на calls(session_id) — ускоряет поиск звонка по идентификатору VoIP-сессии при завершении звонка.

Миграции базы данных:

Миграции хранятся в директории backend/migrations/ и применяются автоматически при старте приложения в порядке сортировки по имени файла:

1. 001_create_users_table.sql — создание таблицы пользователей и индекса;
2. 002_create_calls_table.sql — создание таблицы звонков и базовых индексов;
3. 003_add_webrtc_fields_to_calls.sql — добавление полей session_id, sdp_offer, sdp_answer для поддержки WebRTC и индекса по session_id.

Взаимодействие с базой данных осуществляется через ORM GORM с использованием паттерна «Репозиторий», определенного в слое домена (интерфейсы UserRepository и CallRepository) и реализованного в слое инфраструктуры (postgres/). Настройки пула соединений: максимальное количество открытых соединений — 25, максимальное количество простаивающих соединений — 5. Параметры подключения загружаются из переменных окружения (POSTGRES_HOST, POSTGRES_PORT, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB).

3.7. Способ применения моделей

В процессе разработки клиентской части (фронтенда) применялись генеративные модели искусственного интеллекта. Использование ИИ было обусловлено тем, что команда разработки состоит из специалистов, чья основная экспертиза лежит в области серверной разработки (бэкенд), и наи-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

большую сложность представляла реализация клиентской части на TypeScript и React с интеграцией WebRTC.

Модели применялись следующим образом:

Claude (Anthropic): использовался в интерактивном режиме через среду разработки VS Code для генерации фрагментов программного кода на TypeScript, рефакторинга существующего кода, автоматического исправления ошибок. Модель получала на вход описание требуемой функциональности и существующий контекст кодовой базы, возвращая готовые к интеграции фрагменты кода. С помощью данной модели были реализованы: структура React-приложения (компоненты Call, History, Landing, Login, Register, Layout, CountrySelect, ProtectedRoute), кастомные хуки (useDevices — проверка доступа к микрофону и динамикам, useWebRTC — управление WebRTC-соединением через Twilio Voice SDK), контексты (AuthContext — управление состоянием аутентификации, LocaleContext — переключение языка интерфейса), API-клиент для HTTP-запросов к бэкенду, стили CSS Modules, словари локализации (i18n/translations.ts), данные стран (data/countries.ts).

Cline (DeepSeek provider API): использовался для автодополнения кода, что позволило сократить время написания типовых конструкций: TypeScript-типы и интерфейсы компонентов, обработчики событий ввода и сабмита форм, асинхронные запросы к REST API с обработкой ошибок, анимации и визуальные индикаторы статуса звонка, адаптивные CSS-стили для десктопных и мобильных устройств.

Ни одна из моделей не использовалась для генерации текста программной документации (пояснительной записки, технического задания). Вся документация написана разработчиками вручную в соответствии с требованиями ГОСТ 19 ЕСПД.

Все сгенерированные с помощью ИИ тексты программного кода проходили обязательную проверку и корректировку разработчиком перед включением в итоговый проект. Сгенерированный код подвергался ревью (code review), тестированию и интеграционной проверке в составе проекта. Окончательное решение о включении любого фрагмента кода принимал разработчик.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Курсовой проект представляет собой веб-платформу для международных звонков через браузер. В рамках учебного курсового проекта не проводится расчет экономической эффективности.

4.2. Предполагаемая потребность

Основной целевой аудиторией платформы являются:

1. Частные лица, которым нужно быстро связаться с родственниками и друзьями за рубежом без установки дополнительного программного обеспечения.
2. Небольшие команды и фрилансеры, нуждающиеся в простых международных звонках для рабочих созвонов.
3. Пользователи с базовыми навыками работы с браузером, которым важно, чтобы сервис работал «из коробки».

В отличие от существующих решений (Skype, работа которого была прекращена в 2025 году; Google Voice, требующий установки дополнительного ПО для полного функционала), разрабатываемая платформа обеспечивает полностью браузерный опыт без установки приложений, простой и интуитивный интерфейс, поддержку международных звонков и адаптивный дизайн для всех типов устройств.

4.3. Экономические преимущества разработки по сравнению с аналогами

Для оценки преимуществ создаваемой платформы проведено сравнение ее функциональных характеристик с аналогами — Skype (работа прекращена в 2025 году) и Google Voice. Ссылки на приложения, с которыми проводилось сравнение, находятся в Приложении 2.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Функция	Skype	Google Voice	Наш проект
Работа полностью в браузере без установки ПО	+	-	+
Простой и интуитивный интерфейс	+	+	+
Поддержка международных звонков	+	+	+
Использование WebRTC для прямой передачи аудио	+	-	+
Регистрация и аутентификация пользователей	+	+	+
Просмотр истории звонков	+	+	+
Автоматическая проверка доступа к микрофону/динамикам	+/-	-	+
Отображение статуса звонка и понятных ошибок	+	+	+
Адаптивный дизайн (десктоп + мобильные)	+	+/-	+
Итого (из 9)	8.5	5	9

Таблица 1. Сравнение функциональных характеристик с аналогами

Разрабатываемая платформа превосходит Google Voice по критериям «Работа в браузере без установки ПО» (Google Voice требует установки приложения для мобильных устройств) и «Проверка доступа к микрофону». По сравнению со Skype главное преимущество — это то, что платформа работает и доступна, в то время как Skype прекратил существование в 2025 году. Платформа является единственным полностью браузерным решением среди рассмотренных аналогов.

Ключевые преимущества разрабатываемой платформы:

1. Простота: работает прямо в браузере, минимум настроек, не требует установки.
2. Доступность: подходит для ПК и мобильных устройств с современным браузером.
3. Минимальная нагрузка: опирается на готовый облачный VoIP-сервис (Twilio) без собственного сложного сервера телефонии.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов.
2. ГОСТ 19.102-77: Стадии разработки.
3. ГОСТ 19.103-77: Обозначения программ и программных документов.
4. ГОСТ 19.104-78: Основные надписи.
5. ГОСТ 19.105-78: Общие требования к программным документам.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению.
12. ГОСТ 19.603-78: Общие правила внесения изменений.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом.
14. Официальная документация Go. Электронный ресурс. URL: <https://go.dev/doc/> (дата обращения: 03.11.25)
15. Официальная документация React. Электронный ресурс. URL: <https://react.dev/> (дата обращения: 03.11.25)
16. Официальная документация TypeScript. Электронный ресурс. URL: <https://www.typescriptlang.org/docs/> (дата обращения: 03.11.25)
17. Документация Twilio Voice SDK. Электронный ресурс. URL: <https://www.twilio.com/docs/voice/sdks> (дата обращения: 03.11.25)
18. Документация Gin Web Framework. Электронный ресурс. URL: <https://gin-gonic.com/docs/> (дата обращения: 03.11.25)
19. Документация GORM. Электронный ресурс. URL: <https://gorm.io/docs/> (дата обращения: 03.11.25)
20. Документация PostgreSQL. Электронный ресурс. URL: <https://www.postgresql.org/docs/> (дата обращения: 03.11.25)
21. Документация Docker. Электронный ресурс. URL: <https://docs.docker.com/> (дата обращения: 03.11.25)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

22. Документация Vite. Электронный ресурс. URL: <https://vitejs.dev/guide/> (дата обращения: 03.11.25)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ

Приложение	Ссылка
Skype	https://www.skype.com/
Google Voice	https://voice.google.com/
Twilio	https://www.twilio.com/

Дата обращения: 03.11.25.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ

Таблица 2 — Терминология

Термин	Определение
WebRTC	Технология реального времени, позволяющая веб-браузерам и мобильным приложениям обмениваться аудио- и видеоданными без установки дополнительных плагинов.
VoIP	Технология передачи голоса по IP-сетям, позволяющая совершать телефонные звонки через интернет.
REST API	Архитектурный стиль взаимодействия компонентов распределенного приложения в сети, основанный на HTTP-протоколе.
JWT	JSON Web Token — компактный, URL-безопасный способ представления утверждений (claims) для передачи между двумя сторонами. Используется для аутентификации.
Clean Architecture	Архитектурный подход к построению программного обеспечения, предложенный Робертом Мартином, основанный на разделении ответственности по слоям и направлении зависимостей внутрь к доменному слою.
React	JavaScript-библиотека с открытым исходным кодом для разработки пользовательских интерфейсов, созданная компанией Meta (Facebook).
TypeScript	Язык программирования, являющийся надмножеством JavaScript, добавляющий статическую типизацию.
Go (Golang)	Компилируемый, многопоточный язык программирования с открытым исходным кодом, разработанный компанией Google.
Gin	Высокопроизводительный HTTP-веб-фреймворк для языка Go.
GORM	ORM-библиотека для языка Go, обеспечивающая работу с реляционными базами данных.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Термин	Определение
Docker	Платформа для разработки, доставки и запуска приложений в изолированных контейнерах.
Docker Compose	Инструмент для определения и запуска многоконтейнерных Docker-приложений.
PostgreSQL	Свободная объектно-реляционная система управления базами данных (СУБД).
Twilio	Облачная коммуникационная платформа, предоставляющая API для голосовых вызовов, SMS и других видов связи.
SDP	Session Description Protocol — протокол описания сессии, используемый в WebRTC для согласования параметров медиа-соединения.
bcrypt	Адаптивная криптографическая хеш-функция для безопасного хранения паролей.
Фреймворк	Структурированная платформа, включающая набор инструментов, библиотек и правил, разработанных для упрощения создания и управления программным обеспечением.
Middleware	Программный слой, обеспечивающий взаимодействие между различными компонентами приложения, обрабатывающий запросы до их достижения обработчика.
Миграция БД	Процесс управления изменениями схемы базы данных, обеспечивающий версиюность и воспроизводимость структуры хранения данных.
HTTPS	Расширение протокола HTTP, использующее шифрование TLS/SSL для защищенной передачи данных между клиентом и сервером.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.18-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]